

Merge Sort

Merge sort is a Divide and Conquer sorting algorithm. It divides the array into two halves, recursively sorts each half, and then merges the sorted halves to produce a sorted array.

Working Principle

- Divide the array into two halves
- Continue dividing until each subarray contains only one element
- Merge the subarrays in sorted order
- Repeat until the completed array is sorted.

Example

Input: -

38 27 43 3

Divide: -

38 27 | 43 3

38 | 27 | 43 | 3

Merge: -

38 + 27 → 27 38

43 + 3 → 3 43

Final

27 38 + 3 43

↓

3 27 38 43

Algorithm:-

- Start
- If the array has only one element, return
- Find the middle of the array
- Divide the array into left and right halves
- Recursively sort the left half
- Recursively sort the right half
- Merge two sorted halves
- Merge the two sorted halves
- Stop.

Program:-

```
#include <iostream>
```

```
using namespace std;
```

```
void merge (int arr[], int low, int mid, int high)
```

```
{
```

```
    int temp[100];
```

```
    int i = low;
```

```
    int j = mid + 1;
```

```
    int k = low;
```

```
    while (i <= mid && j <= high)
```

```
    {
```

```
        if (arr[i] <= arr[j])
```

```
            temp[k++] = arr[i++];
```

```
        else
```

```
            temp[k++] = arr[j++];
```

```
    }
```



```
while (i <= mid)
    temp[k++] = arr[i++];
```

```
while (j <= high)
    temp[k++] = arr[j++];
```

```
for (int i = low; i <= high; i++)
    arr[i] = temp[i];
}
```

```
void mergeSort (int arr[], int low, int high)
{
    if (low >= high)
        return;
```

```
    int mid = (low + high) / 2;
```

```
    mergeSort (arr, low, mid);
    mergeSort (arr, mid + 1, high);
```

```
    merge (arr, low, mid, high);
}
```

```
int main ()
{
```

```
    int arr[] = {38, 27, 43, 3};
    int n = 4;
```

```
    mergeSort (arr, 0, n - 1);
```

```
    cout << "sorted array";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```


Quick Sort.

Quick sort is a Divide and Conquer Sorting algorithm. It selects a pivot element, places it at its correct position, and partitions the array into two parts: elements smaller than the pivot and elements greater than pivot.

Working principle

- Choose a pivot element
- Rearrange the array so that
 - Smaller elements are on the left of the pivot
 - Larger elements are on the right of the pivot
- Recursively apply the same process to the left and right subarrays.

Example

Input: 10 7 8 9 1 5

Pivot = 5

Partition:

1 | 5 | 10 7 8 9

Sort right part.

1 | 5 | 7 8 9 10

Output:

1 5 7 8 9 10.

Algorithm

- Start
- Select a pivot element
- Partition the array around the pivot
- Place the pivot in its correct position
- Recursively sort the left subarray
- Recursively sort the right subarray
- Stop if subarray size becomes 0 or 1.

Program:-

```
#include <iostream>
```

```
using namespace std;
```

```
int partition ( int arr[], int low, int high)
```

```
{  
    int pivot = arr[low];
```

```
    int i = low;
```

```
    int j = high;
```

```
    while ( i < j )  
    {
```

```
        while ( arr[i] <= pivot && i < high )
```

```
            i++;
```

```
        while ( arr[j] > pivot )
```

```
            j--;
```

```
        if ( i < j )
```

```
            swap ( arr[i], arr[j] );
```

```
    }
```

```
    swap ( arr[low], arr[j] );
```

```
    return j;
```

```
}
```



```
void quicksort (int arr[], int low, int high)
{
```

```
    if (low < high)
```

```
    {
        int p = partition (arr, low, high);
```

```
        quicksort (arr, low, p-1);
```

```
        quicksort (arr, p+1, high);
```

```
    }
}
```

```
int main()
```

```
{
```

```
    int arr[] = {10, 7, 8, 9, 1, 5}
```

```
    int n = 6;
```

```
    quicksort (arr, 0, n-1);
```

```
    cout << "sorted array: ";
```

```
    for (int i=0; i<n; i++)
```

```
        cout << arr[i] << " ";
```

```
    return 0;
```

```
}
```


Heap sort

Heap sort is a comparison-based sorting algorithm that uses a binary max heap. It first builds a Max heap, then repeatedly swaps the Root with the last element and heapifies the remaining elements until the array is sorted.

Working principle

- Build a max heap from the given array
- Swap the root with the last element
- Reduce the heap size by one
- Heapify the root to restore the max heap
- Repeat until the array is completely sorted.

Algorithm

- Start
- Build a max heap from the array
- Swap the root with the last element
- Reduce the heap size by one
- Heapify the root
- Repeat steps 3-5 until the heap size becomes 1
- Stop.

Example:- Input = 4 10 3 5 1

Build max heap 10 5 3 4 1

Swap 10 and 1 5 3 4 10

Heapify 5 4 3 1 10

Repeat
out put

1 3 4 5 10

Program :-

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify (int arr[], int n, int i)
```

```
{
```

```
    int larger = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[larger])
```

```
        larger = left;
```

```
    if (right < n && arr[right] > arr[larger])
```

```
        larger = right;
```

```
    if (larger != i)
```

```
{
```

```
        swap (arr[i], arr[larger]);
```

```
        heapify (arr, n, larger);
```

```
}
```

```
}
```

```
void heapsort (int arr[], int n)
```

```
{
```

```
    for (int i = n / 2 - 1; i >= 0; i--)
```

```
        heapify (arr, n, i);
```

```
    for (int i = n - 1; i > 0; i--)
```

```
{
```

```
        swap (arr[0], arr[i]);
```

```
        heapify (arr, i, 0);
```

```
}
```



```
int main ()  
{
```

```
int arr[] = { 4, 10, 3, 5, 1 };
```

```
int n = 5;
```

```
rah sort (arr, n);
```

```
cout << "sorted array: " << endl;
```

```
for (int i=0; i<n; i++)
```

```
cout << arr[i] << " ";
```

```
return 0;
```

```
}
```